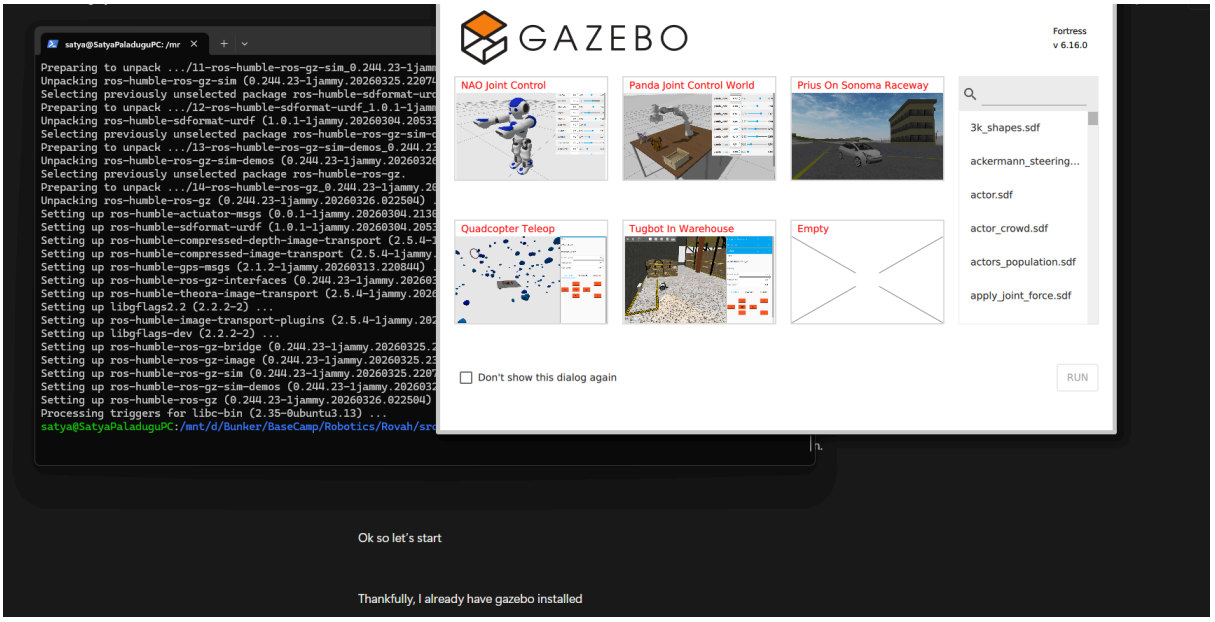
# Category 2

## NOTE: I AM really confused with what goes where and am lacking time due to other shit. I am following ROS2 HUMBLE Documentation and ChatGPT side by side for faster Studying.

### NOTE 2: NOW ONLY I AM TELLING. I LOST IT. WHAT AND ALL. SO MUCH BRAINROT. I SIMPLY UNDERSTOOD TOPIC AND WHAT TO DO AND DELEGATED COPILOT TO DO ALLAT. 🙏

Ok so let's start

Thankfully, I already have gazebo installed




https://youtu.be/kCsUmqDRJt0

Ok I am clueless, So I asked ChatGPT to give me a list of things to work on and learn full.

---

SO NOW LET'S GET STARTED

Here is my Go to Plan.

```
my_ros2_workspace/        # Workspace root (you create this)
├── src/                  # Source directory (required by ROS2)
```

```
│   ├── mini_model_description/    # Your package
│   │   ├── package.xml     # Package metadata
│   │   ├── setup.py        # Python package info
│   │   ├── urdf/           # Your URDF files
│   │   ├── meshes/         # Your mesh files
│   │   └── ...
│   └── (other packages if you add them)
├── build/                  # Created during build (auto-generated)
├── install/                # Created during build (auto-generated)
└── log/                    # Build logs (auto-generated)
```

**First Let's build a proper ROS2 Workspace**

Creating a workspace — ROS 2 Documentation: Humble documentation

```
PS D:\Bunker\BaseCamp\Robotics\Rovah\src> wsl
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/src$ colcon build

Summary: 0 packages finished [0.36s]
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/src$ source install/setup.bash
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/src$
```

Creating a package

Creating a package — ROS 2 Documentation: Humble documentation

**We are told that mini_model_Description itself is a package**

so we willl jbuild it

`colcon build --packages-select my_package`

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/src$ colcon build --packages-select mini_model_description
[0.406s] WARNING:colcon.colcon_core.package_selection:ignoring unknown package 'mini_model_description' in --packages-select

Summary: 0 packages finished [0.26s]
```

I asked chatgpt why. Turns out we need package.xml which describes the package. Makes sense.

## Create your own package

colcon uses the `package.xml` specification defined in REP 149 (format 2 is also supported).

I will make a package called mini_model_Description and copy paste those mesh and urdf into those folders

ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name>

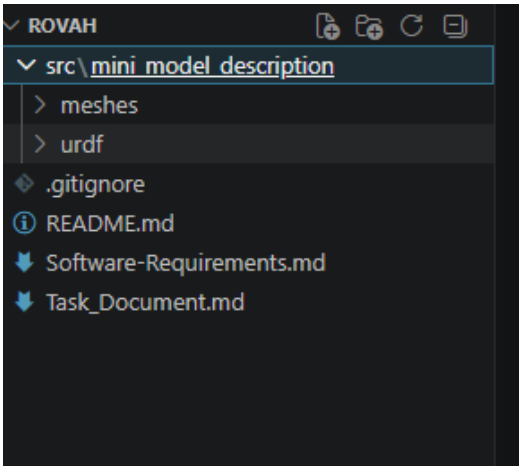
now i will copy and put the urdf and mesh folders

NO WAIT I AM CONFUSED

first lets build a workspace
then let's build package
and then let's keep all mesh urdf files

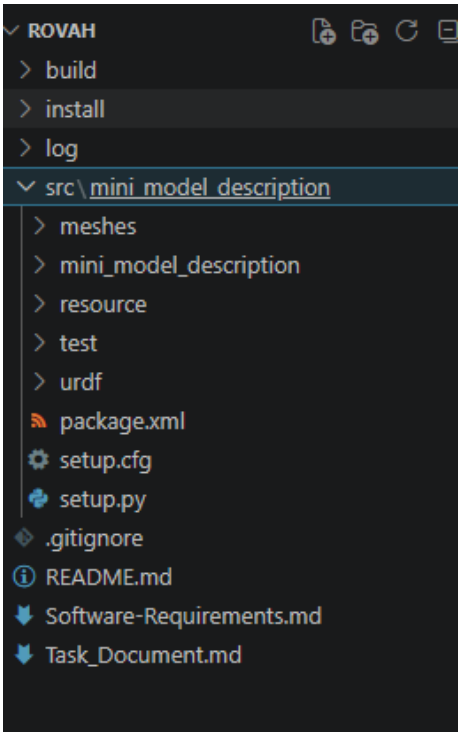
i have realised everytime i keep making changes, i have to keep rebuilding pakage

I hard reset git




now i will do colcon build and then build package

now i will add mesh and urdf



```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ colcon build
Starting >>> mini_model_description
Finished <<< mini_model_description [3.08s]

Summary: 1 package finished [3.52s]
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ source install/setup.bash
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ colcon build --packages-select mini_model_description
Starting >>> mini_model_description
Finished <<< mini_model_description [3.37s]

Summary: 1 package finished [3.67s]
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ # List all packages (should see mini_model_description)
ros2 pkg list
```

```
logging_demo
map_msgs
mecanum_drive_controller
message_filters
micro_ros_msgs
mini_model_description
moveit_common
moveit_configs_utils
```

## `package.xml`

- **What it holds:** Metadata about your ROS2 package
- **What it does:**
  - Tells ROS2 the package name, version, and description
  - Lists all dependencies (what other packages you need)
  - Specifies the build system (CMake, Python, etc.)
  - Used by `colcon` to understand package structure

### `Setup.py`

It functions similarly to a `CMakeLists.txt` file for C++ packages, defining how the package should be built, which files to install, and where to install them

My setup.py do not mention anything about the urdf and .stl files. I

Using URDF with robot_state_publisher (Python) — ROS 2 Documentation: Kilted documentation

I have to modify my setup.py and then build again

ok lets build again, source it

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ source install/setup.bash
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ ls install/mini_model_description/share/mini_model_description/
hook  meshes  package.bash  package.dsv  package.ps1  package.sh  package.xml  package.zsh  urdf
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ ls install/mini_model_description/share/mini_model_description/
hook  meshes  package.bash  package.dsv  package.ps1  package.sh  package.xml  package.zsh  urdf
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ ros2 pkg prefix mini_model_description
/mnt/d/Bunker/BaseCamp/Robotics/Rovah/install/mini_model_description
```

this means, the setup.py has actually realised mesh urdf and evertyhgin else needed by the robot.

https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html

URDF (Unified Robot Description Format) is a file format for specifying the geometry and organization of robots in ROS.

Using URDF with robot_state_publisher (Python) — ROS 2 Documentation: Humble documentation

“This tutorial will show you how to model a walking robot, publish the state as a tf2 message and view the simulation in Rviz”

The fundamental difference between RViz and Gazebo is that Gazebo simulates the physical environment, while RViz visualizes sensor data and robot states.

`tf2` in ROS 2 is **the second-generation transform library**, allowing developers to manage, broadcast, and listen to multiple coordinate frames http://wiki.ros.org/tf2

**robot_state_publisher** is a ROS2 node that:

1. Reads your URDF file (the robot's structure/joints)
2. Listens to joint states (how much each joint is bent/rotated)
3. Broadcasts the robot's pose to the `/tf` (transform) topic

To work with this, we need files in URDF. We have been given files in xacro.

Using Xacro to clean up your code — ROS 2 Documentation: Humble documentation

as the documentation suggests, we can convert each .xacro into urdf

but chatgpt suggested a much cooler method: to do on the fly while running sim

```bash
ros2 run robot_state_publisher robot_state_publisher \
$(ros2 run xacro xacro robot.urdf.xacro)
```

ok now let’s run this

ros2 run robot_state_publisher robot_state_publisher \
$(ros2 run xacro xacro
install/mini_model_description/share/mini_model_description/urdf/<your_file>.xacro)

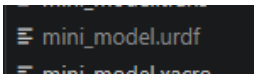
i ran

`ros2 run robot_state_publisher robot_state_publisher \`
`$(ros2 run xacro xacro`
`install\mini_model_description\share\mini_model_description\urdf\mini_model.xacro)`

error:

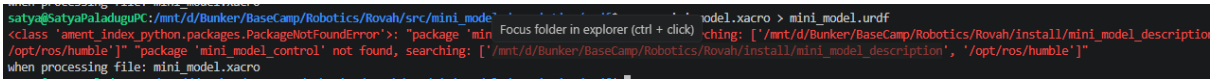
[FATAL] [1775497894.307790193] [robot_state_publisher]: Failed to open URDF file: [ros2run]:: No such file or directory
terminate called after throwing an instance of 'std::system_error'
what():  Failed to open URDF file: [ros2run]:: No such file or directory
[ros2run]: Aborted

leave lemme use xacro and get for
mini_model.xacro i aint doing allat




now let's do https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html

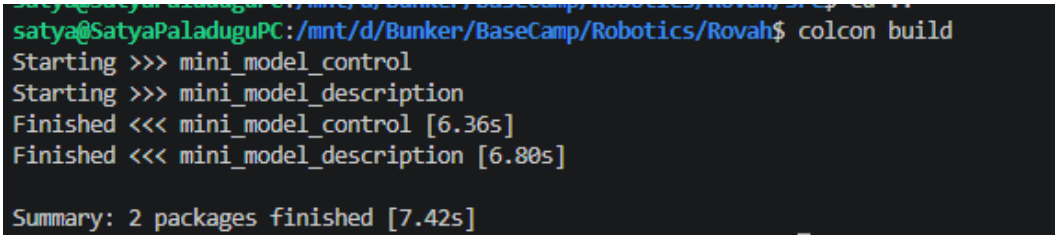
but bro




i can think of only 2 things

open the xacro file, comment the portion out

or make a dummy package for now, later add functionality




YES
IT WORKS

src > mini_model_description > urdf > mini_model.urdf

```xml
<?xml version="1.0" ?>
<!-- ================================================================================ -->
<!-- |    This document was autogenerated by xacro from mini_model.xacro              | -->
<!-- |    EDITING THIS FILE BY HAND IS NOT RECOMMENDED                                | -->
<!-- ================================================================================ -->
<robot name="mini_model">
  <material name="silver">
    <color rgba="0.700 0.700 0.700 1.000"/>
  </material>
  <!-- Sensor Materials -->
  <material name="camera_gray">
    <color rgba="0.5 0.5 0.5 1.0"/>
  </material>
  <material name="lidar_black">
    <color rgba="0.1 0.1 0.1 1.0"/>
  </material>
  <material name="imu_blue">
```

https://github.com/ros/urdf_tutorial/tree/ros2

from this github repository i haveu nderstood these:

there is a display.launch.py shit

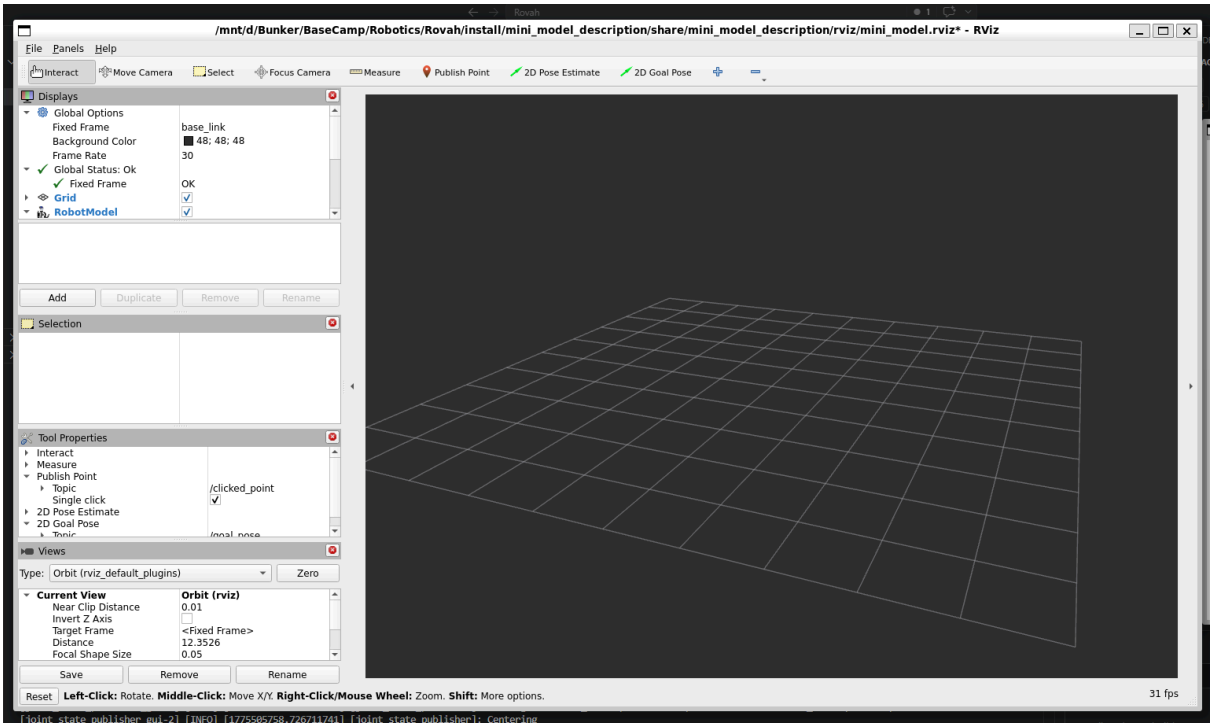
Thefile is a ROS 2 launch script that automates bringing up all the shit you need to see your urdf robot in RViz.

I have asked copilot to make changes to setup.py and make display.launch.py to load up shit in RviZ

turns out i had shit ton of file issues

The links in urdf linking to mesh are malformed. I shall ask copilot to fix that

bro wya 💔



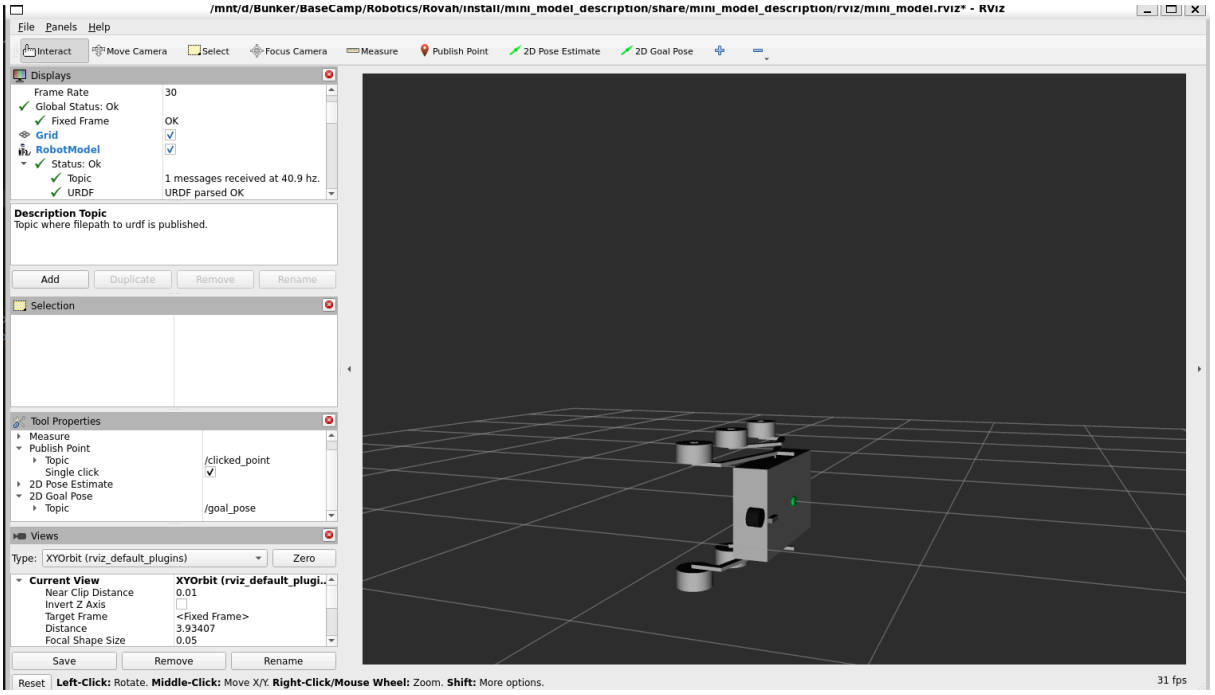

OHHHHHHHHHHHH

I HAVENT SET THE DESCRIPTION SOURCE

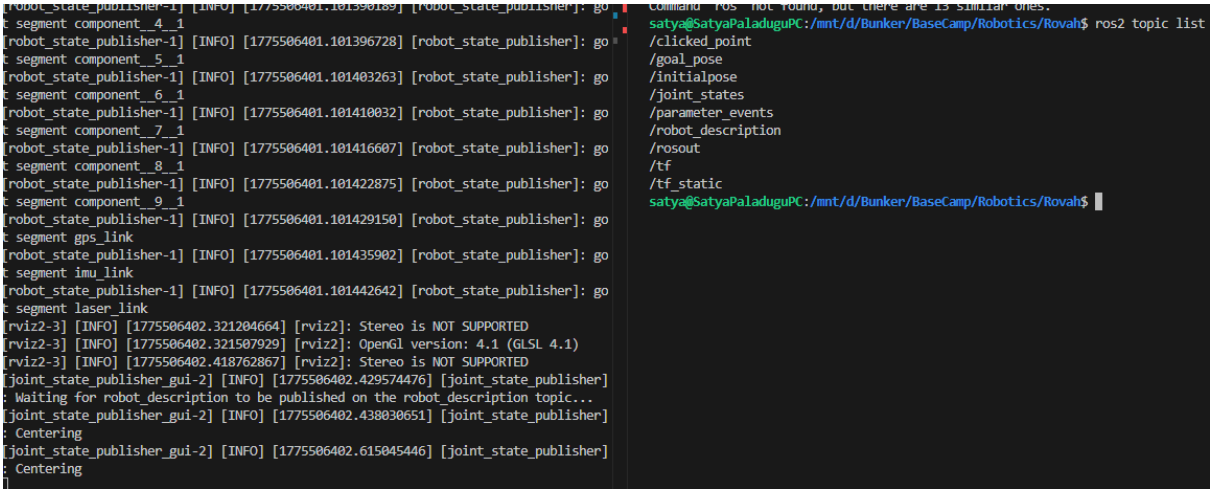
its supposed to be robot_description

in my display.launch.py, we have taken robo_description as the model urdf for mini_model.urdf

and voila



ok wait, let's discuss a little about joint state publisher and robot state publisher




from this very image we can tell that everything being done is a ros2 topic now

crazy right

but no thats not the point

Introducing tf2 — ROS 2 Documentation: Humble documentation
in ROS 2, when we talk about tf we're referring to the system that keeps track of how different parts of your robot are positioned and oriented relative to each other.

AS I HAVE TOLD BEFORE I AM OUT OF TIME FOR READING THE WHOLE DOCUMENTATION SO IM JUST ASKING COPILOT TO SUMMARIZE AND EXPLAIN BETTAH. 🙏

1. **Robot State Publisher**
   - Reads your URDF/Xacro file.
   - For every joint and link, it computes the relative transform (translation + rotation).
   - Publishes these transforms continuously on the `/tf` topic.
2. **Joint State Publisher**
   - Provides the current values of each joint (angles, positions).
   - These values are fed into the robot_state_publisher so transforms update dynamically when joints move.
3. **tf2 Library**
   - ROS 2's transform library that manages all these coordinate frames.

- It builds a tree of frames (e.g., `base_link → shoulder_link → elbow_link → wrist_link`).
- Any node can query tf2 to find the transform between two frames at a given time.

4. **RViz Visualization**
   - RViz subscribes to `/tf` and uses the transform tree to draw your robot correctly.
   - When you move a joint, the transform updates, and RViz redraws the robot in the new pose.

Transforms are published by `joint_state_publisher` (publishes joint_states) and `robot_state_publisher` (computes link transforms from the URDF and publishes them on `/tf` and static frames on `/tf_static`).

ok let's understand what has been done so far

```
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration, Command
from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue
from ament_index_python.packages import get_package_share_dire
ctory


def generate_launch_description():
    # Get package directory
    pkg_share = get_package_share_directory('mini_model_descri
ption')

    # Declare launch arguments
    model_arg = DeclareLaunchArgument(
        'model',
        default_value=os.path.join(pkg_share, 'urdf', 'mini_mo
del.urdf'),
```

```
        description='Path to robot URDF file'
    )
    gui_arg = DeclareLaunchArgument(
        'gui',
        default_value='true',
        description='Flag to enable joint_state_publisher_gui'
    )
    rvizconfig_arg = DeclareLaunchArgument(
        'rvizconfig',
        default_value=os.path.join(pkg_share, 'rviz', 'mini_mo
del.rviz'),
        description='Path to RViz config file'
    )

    # Get arguments
    model = LaunchConfiguration('model')
    gui = LaunchConfiguration('gui')
    rvizconfig = LaunchConfiguration('rvizconfig')

    # Robot State Publisher
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{
            'robot_description': ParameterValue(Command(['cat
', model]), value_type=str)
        }]
    )

    # Joint State Publisher GUI (for manual control)
    joint_state_publisher_gui_node = Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        condition=IfCondition(gui)
    )

    # RViz
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
```

```
        output='screen',
        arguments=['-d', rvizconfig],
    )

    return LaunchDescription([
        model_arg,
        gui_arg,
        rvizconfig_arg,
        robot_state_publisher_node,
        joint_state_publisher_gui_node,
        rviz_node,
    ])
```

**Purpose of display.launch.py**

Declares model argument, defaulting to installed URDF

default_value=os.path.join(pkg_share, 'urdf', 'mini_model.urdf'),

Starts robot_state_publisher and sets robot_description from file content

'robot_description': ParameterValue(Command(['cat ', model]), value_type=str)

Starts joint_state_publisher_gui (manual joint sliders).

Starts RViz with your config file at display.launch.py:59.

- **Launch (starts the nodes)**:

the `robot_state_publisher` and `joint_state_publisher_gui` nodes are started here.

- **URDF/Xacro (defines the link/joint tree)**: mini_model.urdf and the xacro files in the same folder — these define the frames robot_state_publisher publishes

Now this file returns a thing called launch description

this launch description Creating a launch file — ROS 2 Documentation: Foxy documentation

- The `LaunchDescription` is literally the list of actions (nodes, remappings, parameters, event handlers) that ROS 2 will execute when you run `ros2 launch`.
- **When you run** `ros2 launch`
  ROS 2 looks for the `generate_launch_description()` function in your file.

That function must return a `LaunchDescription` object, which tells the launch system what nodes to start and how to configure them.

# Now let’s spawn in gazebo

if i have multiple urdf files and .gazbo files, how do i make use of them and what does each represent?

urdf : The **top-level robot file** that assembles everything.
gazebo: iles that define **simulation-specific behavior**

always open, check package:// or maybe includes=""

doing so would tell

- what components exist
- what dependencies exist

and also make sure the paths are resolved

Ok listen, I am using ros2 humble with gazebo fortress
make sure it is making use of all the files and meshes.
Check the packages
Check the paths and make sure everything is resolved

i hav made some publisher functions. later we will implement for odometer and tf tree

```
import rclpy
from nav_msgs.msg import Odometry
from rclpy.node import Node


class DummyOdomPublisher(Node):
    def __init__(self):
        super().__init__('dummy_odom_publisher')
        self.publisher_ = self.create_publisher(Odometry, '/od
om', 10)
```

```
        self.timer_ = self.create_timer(0.1, self.publish_odo
m)

    def publish_odom(self):
        msg = Odometry()
        now = self.get_clock().now().to_msg()
        msg.header.stamp = now
        msg.header.frame_id = 'odom'
        msg.child_frame_id = 'base_link'

        msg.pose.pose.orientation.w = 1.0
        msg.twist.twist.linear.x = 0.0
        msg.twist.twist.angular.z = 0.0

        self.publisher_.publish(msg)


def main(args=None):
    rclpy.init(args=args)
    node = DummyOdomPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

and

```
import rclpy
from geometry_msgs.msg import TransformStamped
from rclpy.node import Node
from tf2_ros import TransformBroadcaster
```

```
class DummyTfBroadcaster(Node):
    def __init__(self):
        super().__init__('dummy_tf_broadcaster')
        self.tf_broadcaster = TransformBroadcaster(self)
        self.timer_ = self.create_timer(0.1, self.publish_tran
sform)

    def publish_transform(self):
        transform = TransformStamped()
        transform.header.stamp = self.get_clock().now().to_msg
()
        transform.header.frame_id = 'odom'
        transform.child_frame_id = 'base_link'

        transform.transform.translation.x = 0.0
        transform.transform.translation.y = 0.0
        transform.transform.translation.z = 0.0
        transform.transform.rotation.w = 1.0

        self.tf_broadcaster.sendTransform(transform)


def main(args=None):
    rclpy.init(args=args)
    node = DummyTfBroadcaster()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

added to setup.py

```
entry_points={
    'console_scripts': [
        'dummy_odom_publisher = mini_model_description.dummy_odom_publisher:main',
        'dummy_tf_broadcaster = mini_model_description.dummy_tf_broadcaster:main',
    ],
},
```

working of my spawn_gazebo.py

```
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunc
hDescription, LogInfo, SetEnvironmentVariable
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDesc
riptionSource
from launch.substitutions import Command, EnvironmentVariable,
LaunchConfiguration
from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue
from ament_index_python.packages import (
    PackageNotFoundError,
    get_package_prefix,
    get_package_share_directory,
)


def _odom_publisher_node(enabled):
    return Node(
        package='mini_model_description',
        executable='odom_publisher',
        name='odom_publisher',
        output='screen',
        condition=IfCondition(enabled),
    )


def _tf_broadcaster_node(enabled):
    return Node(
        package='mini_model_description',
        executable='tf_broadcaster',
        name='tf_broadcaster',
```

```
        output='screen',
        condition=IfCondition(enabled),
    )


def generate_launch_description():
    pkg_share = get_package_share_directory('mini_model_descri
ption')
    pkg_prefix = get_package_prefix('mini_model_description')
    share_root = os.path.join(pkg_prefix, 'share')

    model_arg = DeclareLaunchArgument(
        'model',
        default_value=os.path.join(pkg_share, 'urdf', 'mini_mo
del_ignition.xacro'),
        description='Path to robot model xacro/urdf for Gazebo
Fortress',
    )
    entity_name_arg = DeclareLaunchArgument(
        'entity_name',
        default_value='mini_model_ignition',
        description='Name of spawned model in Gazebo',
    )
    start_gz_arg = DeclareLaunchArgument(
        'start_gz',
        default_value='true',
        description='Start Gazebo (ros_gz_sim) from this launc
h',
    )
    gz_args_arg = DeclareLaunchArgument(
        'gz_args',
        default_value='-r empty.sdf',
        description='Arguments forwarded to ros_gz_sim gz_sim.
launch.py',
    )
    use_sim_time_arg = DeclareLaunchArgument(
        'use_sim_time',
        default_value='true',
        description='Use simulation clock',
    )
    publish_odom_arg = DeclareLaunchArgument(
```

```
        'publish_odom',
        default_value='true',
        description='Run odom publisher node',
    )
    publish_tf_arg = DeclareLaunchArgument(
        'publish_tf',
        default_value='true',
        description='Run TF broadcaster node',
    )
    spawn_x_arg = DeclareLaunchArgument(
        'spawn_x',
        default_value='0.0',
        description='Initial spawn X position (meters)',
    )
    spawn_y_arg = DeclareLaunchArgument(
        'spawn_y',
        default_value='0.0',
        description='Initial spawn Y position (meters)',
    )
    spawn_z_arg = DeclareLaunchArgument(
        'spawn_z',
        default_value='0.32',
        description='Initial spawn Z position (meters)',
    )
    spawn_roll_arg = DeclareLaunchArgument(
        'spawn_roll',
        default_value='1.57079632679',
        description='Initial spawn roll in radians (+90 deg to
align CAD up-axis with Gazebo Z-up)',
    )
    spawn_pitch_arg = DeclareLaunchArgument(
        'spawn_pitch',
        default_value='0.0',
        description='Initial spawn pitch in radians',
    )
    spawn_yaw_arg = DeclareLaunchArgument(
        'spawn_yaw',
        default_value='-1.57079632679',
        description='Initial spawn yaw in radians (-90 deg = c
lockwise)',
    )
```

```
    model = LaunchConfiguration('model')
    entity_name = LaunchConfiguration('entity_name')
    start_gz = LaunchConfiguration('start_gz')
    gz_args = LaunchConfiguration('gz_args')
    use_sim_time = LaunchConfiguration('use_sim_time')
    publish_odom = LaunchConfiguration('publish_odom')
    publish_tf = LaunchConfiguration('publish_tf')
    spawn_x = LaunchConfiguration('spawn_x')
    spawn_y = LaunchConfiguration('spawn_y')
    spawn_z = LaunchConfiguration('spawn_z')
    spawn_roll = LaunchConfiguration('spawn_roll')
    spawn_pitch = LaunchConfiguration('spawn_pitch')
    spawn_yaw = LaunchConfiguration('spawn_yaw')

    # Ensure Gazebo Fortress can resolve model://mini_model_de
scription/... URIs.
    resource_path_value = [
        share_root,
        os.pathsep,
        pkg_share,
        os.pathsep,
        os.path.join(pkg_share, 'meshes'),
        os.pathsep,
        os.path.join(pkg_share, 'urdf'),
    ]

    gz_resource_path = SetEnvironmentVariable(
        name='GZ_SIM_RESOURCE_PATH',
        value=resource_path_value + [
            os.pathsep,
            EnvironmentVariable('GZ_SIM_RESOURCE_PATH', defaul
t_value=''),
        ],
    )
    ign_resource_path = SetEnvironmentVariable(
        name='IGN_GAZEBO_RESOURCE_PATH',
        value=resource_path_value + [
            os.pathsep,
            EnvironmentVariable('IGN_GAZEBO_RESOURCE_PATH', de
fault_value=''),
```

```
        ],
    )

    robot_description = ParameterValue(Command(['xacro', ' ',
model]), value_type=str)

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{
            'robot_description': robot_description,
            'use_sim_time': use_sim_time,
        }],
        output='screen',
    )

    try:
        get_package_share_directory('ros_gz_bridge')
        clock_bridge_node = Node(
            package='ros_gz_bridge',
            executable='parameter_bridge',
            arguments=['/clock@rosgraph_msgs/msg/Clock[gz.msg
s.Clock'],
            output='screen',
        )
    except PackageNotFoundError:
        clock_bridge_node = LogInfo(
            msg='Package ros_gz_bridge not found. /clock bridg
e will not run.'
        )

    try:
        ros_gz_sim_share = get_package_share_directory('ros_gz
_sim')
        gz_launch = IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(ros_gz_sim_share, 'launch', 'gz_s
im.launch.py')
            ),
            condition=IfCondition(start_gz),
            launch_arguments={'gz_args': gz_args}.items(),
```

```
        )

        spawn_entity = Node(
            package='ros_gz_sim',
            executable='create',
            arguments=[
                '-name', entity_name,
                '-topic', '/robot_description',
                '-x', spawn_x,
                '-y', spawn_y,
                '-z', spawn_z,
                '-R', spawn_roll,
                '-P', spawn_pitch,
                '-Y', spawn_yaw,
            ],
            output='screen',
        )
    except PackageNotFoundError:
        gz_launch = LogInfo(
            msg='Package ros_gz_sim not found. Install ros-hum
ble-ros-gz-sim.'
        )
        spawn_entity = LogInfo(
            msg='Skipping robot spawn because ros_gz_sim is un
available.'
        )

    return LaunchDescription([
        model_arg,
        entity_name_arg,
        start_gz_arg,
        gz_args_arg,
        use_sim_time_arg,
        publish_odom_arg,
        publish_tf_arg,
        spawn_x_arg,
        spawn_y_arg,
        spawn_z_arg,
        spawn_roll_arg,
        spawn_pitch_arg,
        spawn_yaw_arg,
```

```
        gz_resource_path,
        ign_resource_path,
        gz_launch,
        robot_state_publisher_node,
        clock_bridge_node,
        spawn_entity,
        _odom_publisher_node(publish_odom),
        _tf_broadcaster_node(publish_tf),
    ])
```

Funny thing

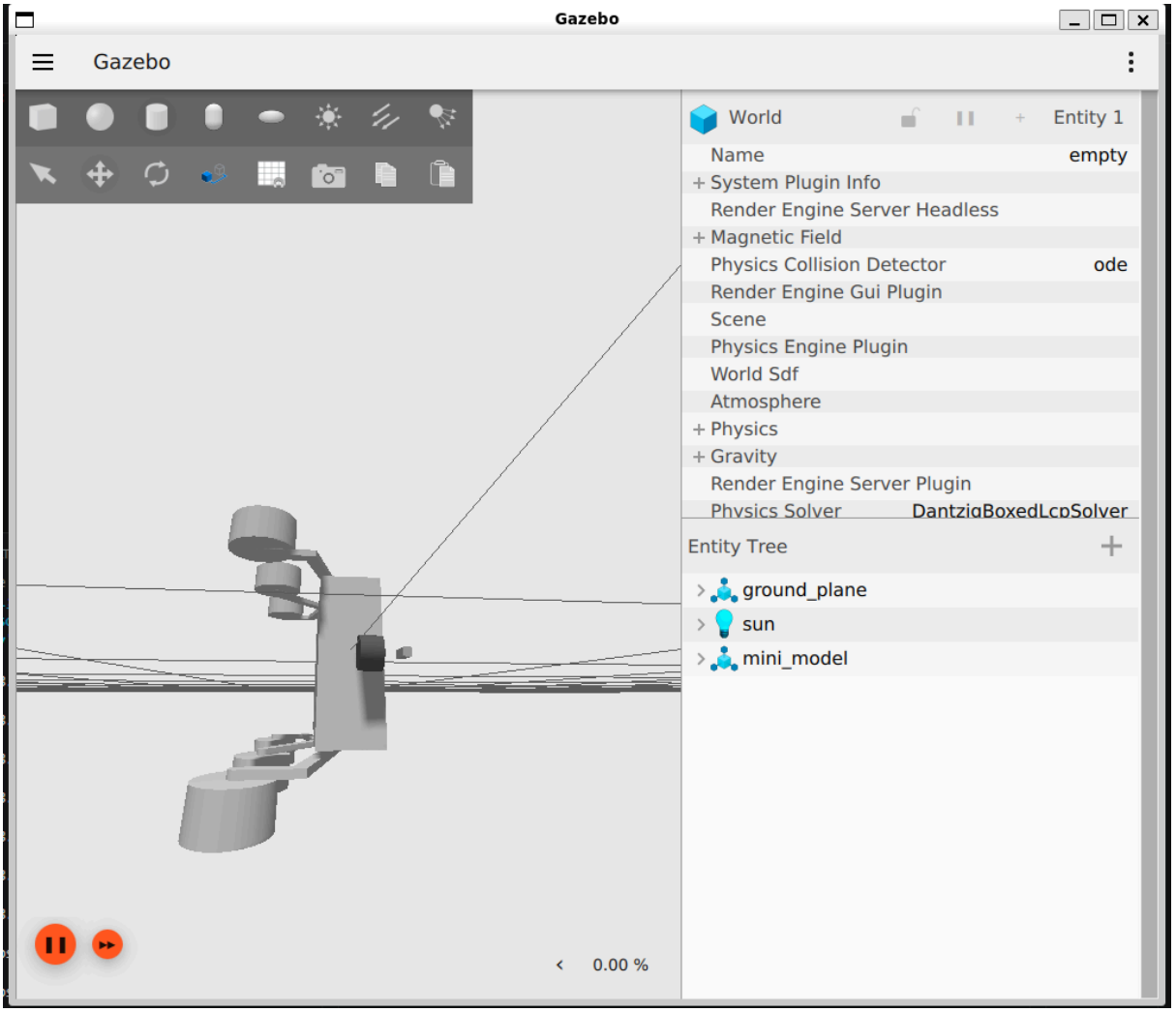
my wsl wasnt able to present the gui. it was constantly crashing

i had to run export LIBGL_ALWAYS_SOFTWARE=1 to mae

Ok peace

ROVER HAS LOADED BAHAHAHAHHAHA

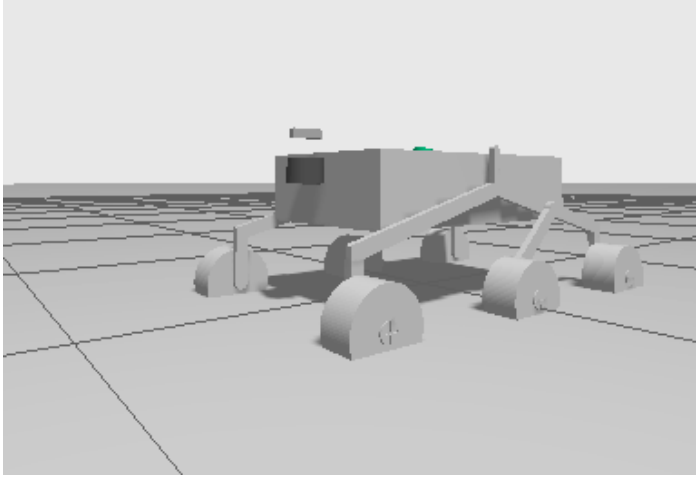
but bro is fallen down




i asked copilot to make changes to the spawn gazebo file to make it spawn properly

it added some launch arguments

```python
def generate_launch_description():
        default_value='true',
        description='Run TF broadcaster node',
    )
    spawn_x_arg = DeclareLaunchArgument(
        'spawn_x',
        default_value='0.0',
        description='Initial spawn X position (meters)',
    )
    spawn_y_arg = DeclareLaunchArgument(
        'spawn_y',
        default_value='0.0',
        description='Initial spawn Y position (meters)',
    )
    spawn_z_arg = DeclareLaunchArgument(
        'spawn_z',
        default_value='0.42',
        description='Initial spawn Z position (meters)',
    )
    spawn_roll_arg = DeclareLaunchArgument(
        'spawn_roll',
        default_value='1.57079632679',
        description='Initial spawn roll in radians (+90 deg to align CAD up-axis with Gazebo Z-up)
    )
    spawn_pitch_arg = DeclareLaunchArgument(
        'spawn_pitch',
        default_value='0.0',
        description='Initial spawn pitch in radians',
    )
    spawn_yaw_arg = DeclareLaunchArgument(
        'spawn_yaw',
        default_value='-1.57079632679',
        description='Initial spawn yaw in radians (-90 deg = clockwise)',
    )
```



now its time to Configure and run a differential drive controller using `ros2_control`.

remember earlier we made a dummy mini_model_control to make the xacro file convert to urdf (to fix dependency issue)

here we will give functionality to that package, build it again and see how it is

```
https://github.com/ros-controls/ros2_control_demos
```

diff_drive_controller — ROS2_Control: Humble Apr 2026 documentation

what is differential drive

Differential Drive Robot: Kinematics & Speed Control - Zbotic

A differential drive robot uses two independently driven wheels mounted on a common axis. Steering is achieved entirely by varying the relative speeds of the two wheels — no dedicated steering mechanism is needed.

this reminds me of those two wheeled robots when they needed to steer. one of the wheel was off and the other rotated

AYE GO DA NO TIME

ChatGPT chat: https://chatgpt.com/share/69d24515-4af8-8324-8d09-9ad4bb52bc1e